



Python Programming Intern

Name: Amima Batool

Starting Date: August 26, 2026

Ending Date: September 15, 2026

Project no. : First

Project name: To-Do List Application

To-Do List Application

Introduction:

The To-Do List Application is a command-line task management application developed using Python. This application allows users to manage their daily tasks. Users can add tasks, view all tasks with their status, remove tasks and mark tasks as completed. This project is developed as the part of the Python programming internship at DecodeLabs.

This projects helps to practice fundamental Python concepts and build a practical beginner level application. This project provides practical hand-on experience in developing a Python application while applying concepts like lists, dictionary, functions, loops, conditional statements and error handlings.

Tools & Technologies Used:

- VS Code
- Python
- GitHub

Objectives of the Project:

The main objectives of the To-Do List Application are:

- To practice fundamental Python prpgramming concepts.
- To learn how to store and manage multiple tasks.
- To understand how functions are used to make program logic.
- To handle user input, output and error handling.
- To practice loops and conditional statements.
- To gain hand-on experience in building a project.
- To develop a command-line application.

Features:

Following are the features of the To-Do List Application:

1. Add a New Task:

Users can add a new task to their to-do list by task name with details, it they want to.

2. View All Tasks:

Users can view all stored tasks along with their status.

3. Remove a Task:

Users can remove a listed task from a to-do list.

4. Mark a Task as Completed:

Users can mark a task as completed after finishing it.

5. Exit the Application:

Users can exit the application through the exit option given in menu.

Python Concepts Used:

Following fundamental Python Concepts are used in this project:

➤ Lists:

Lists are used to store and manage multiple tasks in the applications.

```
# List to store tasks
my_tasks = [] # List to store tasks
```

Here, *append()* is the list method to store the tasks in the list.

```
#Function to Add a New Task
def add_task():
    task = input("Enter a new task: ")
    ✦ my_tasks.append({"Task": task, "Status": "Pending.."})
    print("New Task Added Successfully!")
    print()
```

➤ Dictionary:

Dictionaries are used to organize the information related to task. As, here we used dictionary to organize task and its status.

```
#Function to Add a New Task
def add_task():
    task = input("Enter a new task: ")
    ✨ my_tasks.append({"Task": task, "Status": "Pending.."})
    print("New Task Added Successfully!")
    print()
```

➤ Functions:

Functions are used to divide the program into reusable sections. We create different user defined functions to perform different operations like adding a task, viewing tasks, removing a task and marking task as completed.

```
#----FUNCTION DEFINATIONS-----

#Function to Add a New Task
def add_task():
    task = input("Enter a new task: ")
    my_tasks.append({"Task": task, "Status": "Pending.."})
    print("New Task Added Successfully!")
    print()

#Function to View all Tasks
def view_tasks():
    print("Your todo List: ")
    if len(my_tasks) == 0:
        print("No Pending Tasks! ")
    else:
        for index, task in enumerate(my_tasks, 1):
            print(f"{index}: {task['Task']} - {task['Status']}")
    print()
```

```

#Function to Remove a Task
def remove_task():
    if len(my_tasks) == 0:
        print("List is Empty! ")
    else:
        search_index = int(input("Enter the Task Number you want to remove from the list:")) - 1
        if 0 <= search_index < len(my_tasks):
            removed_task = my_tasks.pop(search_index)
            print(f"Task Removed: {removed_task['Task']}")
        else:
            print("Invalid Task Number...")
    print()

#Function to Mark a Task as Completed
def mark_completed():
    if len(my_tasks) == 0:
        print("List is Empty! ")
    else:
        search_index = int(input("Enter the Task Number that you want to mark as completed:")) - 1
        if 0 <= search_index < len(my_tasks):
            my_tasks[search_index]['Status'] = 'completed'
            print(f"Task {my_tasks[search_index]['Task']} has been marked as completed..")
        else:
            print("Invalid Task Number...")
    print()

```

➤ Loops:

Loops are used to repeatedly display the menu and allow the users to perform multiple task operations without restarting the whole program.

➤ Conditional Statements:

Conditional statements are used to make decisions based on the user selection.

Working of the Program:

The To-Do List Application works in a step-by-step manner to perform basic task management operations. The execution process of the program is as follows:

1. Display Menu:

When the program starts, a menu is displayed in the terminal. The user selects an option according to its need.

```

#Function to display a menu
def menu():
    while(True):
        print("----Main Menu----")
        print("1. Add a New Task")
        print("2. View all Tasks")
        print("3. Remove a task")
        print("4. Mark a task as Completed")
        print("5. Exit")
        print()

        choice = input("Enter your choice: ")
        print()
        if choice == "1":
            add_task()
        elif choice == "2":
            view_tasks()
        elif choice == "3":
            remove_task()
        elif choice == "4":
            mark_completed()
        elif choice == "5":
            print("Exiting the application....")
            print()
            exit()
        else:
            print("Invalid choice! Try again!!")
            print()

menu()

```

A `menu()` function is created, which is called when program is run. Loop is used to print menu every time until the user choose to exit the program from the menu.

2. Select an Option:

When the menu displayed in the terminal, there are five different option for user to choose.

```
PS C:\Users\amima\OneDrive\Desktop\VS CODE folder> python todo.py
---Main Menu---
1. Add a New Task
2. View all Tasks
3. Remove a task
4. Mark a task as Completed
5. Exit

Enter your choice: █
```

- **Add a New Task:**

User defined function `add_task()` is created to add a new task in the list. The list method `append()` is used to store the task in the list. In the `append()`, the dictionary is used to store task with its status.

```
#Function to Add a New Task
def add_task():
    task = input("Enter a new task: ")
    my_tasks.append({"Task": task, "Status": "Pending.."})
    print("New Task Added Successfully!")
    print()
```

The user selects the option from the menu to add a new task and enters the task. The task will be store along its status.

```
PS C:\Users\amima\OneDrive\Desktop\VS CODE folder> python todo.py
---Main Menu---
1. Add a New Task
2. View all Tasks
3. Remove a task
4. Mark a task as Completed
5. Exit

Enter your choice: 1

Enter a new task: Code Program
New Task Added Successfully!
```

```
---Main Menu---
1. Add a New Task
2. View all Tasks
3. Remove a task
4. Mark a task as Completed
5. Exit

Enter your choice: 1

Enter a new task: Buy milk
New Task Added Successfully!
```

- **View All Tasks:**

User defined function `view_tasks()` is created to view all the tasks stored in the list. In this function, if-conditional is used to check whether the list is empty or not. If list is not empty, all tasks in the list will view.

```
#Function to View all Tasks
def view_tasks():
    print("Your todo List: ")
    if len(my_tasks) == 0:
        print("No Pending Tasks! ")
    else:
        for index, task in enumerate(my_tasks, 1):
            print(f"{index}: {task['Task']} - {task['Status']}")
        print()
```

The user selects the option from the menu to view all tasks. The task list along there status will appear.

```
----Main Menu----
1. Add a New Task
2. View all Tasks
3. Remove a task
4. Mark a task as Completed
5. Exit

Enter your choice: 2

Your todo List:
1: Code Program - Pending..
2: Buy milk - Pending..
```

- **Remove a Task:**

User defined function ***remove_task()*** is created to remove a task from the list. In this function, if-condition is used to check rather the list is empty or not. If list is not empty, the required task will be remove from the list and display that which task is removed.

```
#Function to Remove a Task
def remove_task():
    if len(my_tasks) == 0:
        print("List is Empty! ")
    else:
        search_index = int(input("Enter the Task Number you want to remove from the list:")) - 1
        if 0<= search_index < len(my_tasks):
            removed_task = my_tasks.pop(search_index)
            print(f"Task Removed: {removed_task['Task']}")
        else:
            print("Invalid Task Number...")
    print()
```

The user selects the option from the menu to remove the task. The task list along there status will be removed.

```
----Main Menu----
1. Add a New Task
2. View all Tasks
3. Remove a task
4. Mark a task as Completed
5. Exit

Enter your choice: 3

Enter the Task Number you want to remove from the list: 2
Task Removed: Buy milk
```

```

----Main Menu----
1. Add a New Task
2. View all Tasks
3. Remove a task
4. Mark a task as Completed
5. Exit

```

Enter your choice: 2

```

Your todo List:
1: Code Program - Pending..

```

- **Mark a task as completed:**

User defined function *mark_completed()* is created to mark the pending task as completed. In this function, loop is used to check rather the list is empty or not. If list is not empty, the task will mark as completed.

```

#Function to Mark a Task as Completed
def mark_completed():
    if len(my_tasks) == 0:
        print("List is Empty! ")
    else:
        search_index = int(input("Enter the Task Number that you want to mark as completed:")) - 1
        if 0 <= search_index < len(my_tasks):
            my_tasks[search_index]['Status'] = 'Completed'
            print(f"Task {my_tasks[search_index]['Task']} has been marked as completed..")
        else:
            print("Invalid Task Number...")
    print()

```

The user selects the option from the menu to mark the task as completed. The status of the task will change into completed.

```

----Main Menu----
1. Add a New Task
2. View all Tasks
3. Remove a task
4. Mark a task as Completed
5. Exit

```

Enter your choice: 4

```

Enter the Task Number that you want to mark as completed: 1
Task Code Program has been marked as completed..

```

```

----Main Menu----
1. Add a New Task
2. View all Tasks
3. Remove a task
4. Mark a task as Completed
5. Exit

```

Enter your choice: 2

```

Your todo List:
1: Code Program - Completed

```

3. Exit the Application:

To exit the application, the built-in *exit()* function is used.

```

        mark_completed()
    elif choice == "5":
        print("Exiting the application....")
        print()
        exit()
    else:
        print("Invalid choice! Try again!!")

```

```

----Main Menu----
1. Add a New Task
2. View all Tasks
3. Remove a task
4. Mark a task as Completed
5. Exit

```

```
Enter your choice: 5
```

```
Exiting the application....
```

```
PS C:\Users\amima\OneDrive\Desktop\VS CODE folder> |
```

Application Flow:

The overall basic workflow of the application is as follow:

Start → Display menu() function → User Selects an option → Add/View/Remove/Mark task → Perform operation → Display menu again → Exit Application → End.

Code:

List to store tasks

```
my_tasks = [] # List to store tasks
```

#-----FUNCTION DEFINITIONS-----

#Function to Add a New Task

def add_task():

```
    task = input("Enter a new task: ")
```

```
    my_tasks.append({"Task": task, "Status": "Pending.."})
```

```
    print("New Task Added Successfully!")
```

```
    print()
```

#Function to View all Tasks

def view_tasks():

```
    print("Your todo List: ")
```

```
    if len(my_tasks) == 0:
```

```
        print("No Pending Tasks! ")
```

```
    else:
```

```
        for index, task in enumerate(my_tasks, 1):
```

```
            print(f"{index}: {task['Task']} - {task['Status']}")
```

```
print()
```

#Function to Remove a Task

```
def remove_task():
```

```
    if len(my_tasks) == 0:
```

```
        print("List is Empty! ")
```

```
    else:
```

```
        search_index = int(input("Enter the Task Number you want to remove from the list:")) - 1
```

```
        if 0 <= search_index < len(my_tasks):
```

```
            removed_task = my_tasks.pop(search_index)
```

```
            print(f"Task Removed: {removed_task['Task']}")
```

```
        else:
```

```
            print("Invalid Task Number...")
```

```
    print()
```

#Function to Mark a Task as Completed

```
def mark_completed():
```

```
    if len(my_tasks) == 0:
```

```
        print("List is Empty! ")
```

```
    else:
```

```
        search_index = int(input("Enter the Task Number that you want to mark as completed:")) - 1
```

```
        if 0 <= search_index < len(my_tasks):
```

```
            my_tasks[search_index]['Status'] = 'Completed'
```

```
            print(f"Task {my_tasks[search_index]['Task']} has been marked as completed..")
```

```
        else:
```

```
            print("Invalid Task Number...")
```

```
    print()
```

#Function to display a menu

```
def menu():
```

```
    while(True):
```

```
        print("----Main Menu----")
```

```
        print("1. Add a New Task")
```

```
        print("2. View all Tasks")
```

```
        print("3. Remove a task")
```

```
        print("4. Mark a task as Completed")
```

```
        print("5. Exit")
```

```
    print()
```

```
    choice = input("Enter your choice: ")
```

```

print()
if choice == "1":
    add_task()
elif choice == "2":
    view_tasks()
elif choice == "3":
    remove_task()
elif choice == "4":
    mark_completed()
elif choice == "5":
    print("Exiting the application....")
    print()
    exit()
else:
    print("Invalid choice! Try again!!")
    print()

```

menu() #Function call

Sample Output:

```

PS C:\Users\amima\OneDrive\Desktop\VS CODE folder> python todo.py
----Main Menu----
1. Add a New Task
2. View all Tasks
3. Remove a task
4. Mark a task as Completed
5. Exit

Enter your choice: 1

Enter a new task: Code Program
New Task Added Successfully!

----Main Menu----
1. Add a New Task
2. View all Tasks
3. Remove a task
4. Mark a task as Completed
5. Exit

Enter your choice: 1

Enter a new task: Buy milk
New Task Added Successfully!

----Main Menu----
1. Add a New Task
2. View all Tasks
3. Remove a task
4. Mark a task as Completed
5. Exit

Enter your choice: 2

Your todo List:
1: Code Program - Pending..
2: Buy milk - Pending..

----Main Menu----

```

```

Enter your choice: 2

Your todo List:
1: Code Program - Pending..
2: Buy milk - Pending..

----Main Menu----
1. Add a New Task
2. View all Tasks
3. Remove a task
4. Mark a task as Completed
5. Exit

Enter your choice: 3

Enter the Task Number you want to remove from the list: 2
Task Removed: Buy milk

----Main Menu----
1. Add a New Task
2. View all Tasks
3. Remove a task
4. Mark a task as Completed
5. Exit

Enter your choice: 2

Your todo List:
1: Code Program - Pending..

----Main Menu----
1. Add a New Task
2. View all Tasks
3. Remove a task
4. Mark a task as Completed
5. Exit

```

```

Your todo List:
1: Code Program - Pending..

----Main Menu----
1. Add a New Task
2. View all Tasks
3. Remove a task
4. Mark a task as Completed
5. Exit

Enter your choice: 4

Enter the Task Number that you want to mark as completed: 1
Task Code Program has been marked as completed..

----Main Menu----
1. Add a New Task
2. View all Tasks
3. Remove a task
4. Mark a task as Completed
5. Exit

Enter your choice: 2

Your todo List:
1: Code Program - Completed

----Main Menu----
1. Add a New Task
2. View all Tasks
3. Remove a task
4. Mark a task as Completed
5. Exit

Enter your choice: 5

Exiting the application....

PS C:\Users\amima\OneDrive\Desktop\VS CODE folder>

```

Testing:

The application was tested by performing each operation:

Sr.	Task	Result	Status
1.	Add a new Task	Task was added successfully.	Passed
2.	View all Tasks	Tasks were viewed successfully along with their status.	Passed
3.	Remove a Task	Task was removed successfully.	Passed
4.	Mark as completed	Task was successfully marked as completed.	Passed
5.	Exit Application	Terminate Program successfully.	Passed

Conclusion:

The To-Do List Application project successfully demonstrates the use of basic Python fundamental programming concepts in a real-world application. The project successfully provides basic task management functionality, including adding, viewing, completing, and removing tasks.

Developing this application helped strengthen my understanding of Python programming, logical problem-solving, user input handling, project organization, and GitHub usage.

GitHub Repository:

[AmimaBatoool/Python Todo-List Application: A simple Python To-Do List application for daily tasks, built as my first internship project at DecodeLabs.](#)

Project status: Completed.